

FLUX

A Visual Data Pipeline Builder

SPL-II: SWE 4404 SRS Report

TEAM 5

Mahmudul Hossain Samir (230042101)
Ifham Bin Hossain (230042112)
Shaikh Muhaiminul Hasan (230042113)

July 26, 2026

Contents

Level 0 – Context Diagram (System Boundaries) **2**

Level 1 – Major Subsystems Flows **3**

Level 2 – Process 3: Build & Execute Pipeline **4**

Level 2 – Process 4: Convert Pipeline **5**

Complete Data Flow Diagram (All Levels) **6**

Three-Layered Architecture **7**

Appendix: Data Store Legend **8**

References **8**

Level 0 – Context Diagram (System Boundaries)

This diagram shows FLUX from the outside: who uses it, and what data moves in and out of the system as a whole.

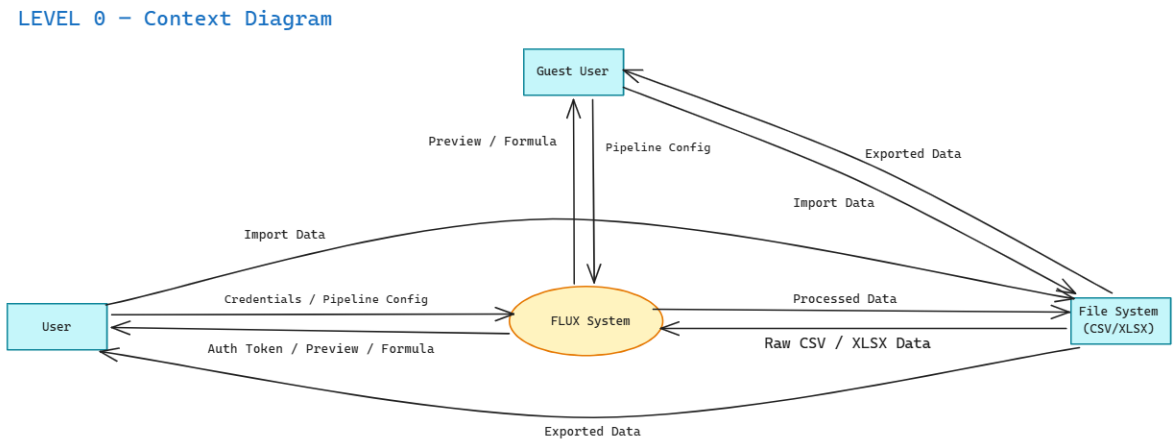


Figure 1: Level 0 – Context Diagram

Primary Data & Interaction Flows:

- **Registered User:** A user sends their login details and their pipeline design (the boxes and connections they built on the canvas) to FLUX. In return, FLUX sends back a login token, a preview of the results, and the matching Excel formula.
- **Guest User:** A guest can send a pipeline design to FLUX just to see a live preview and formula, without logging in. FLUX sends back the same kind of preview and formula, but a guest's work is never saved.
- **File Import/Export:** Both users and guests can bring in their own data files (CSV/XLSX) directly from the file system, and can also export a processed result back out as a file once they are done.
- **FLUX and the File System:** Behind the scenes, FLUX reads the raw data a user uploaded and, after processing it, writes the finished, transformed data back out so it can be downloaded.

Note: The Guest User role shown above was introduced after feedback from the initial SRS presentation. A guest has access to every facility a registered user does, with the one exception that a guest cannot save their pipeline as a template.

Level 1 – Major Subsystems Flows

This breakdown opens up the FLUX box from Level 0 into its four main internal processes, and shows how data hands off between them and the places data is stored.

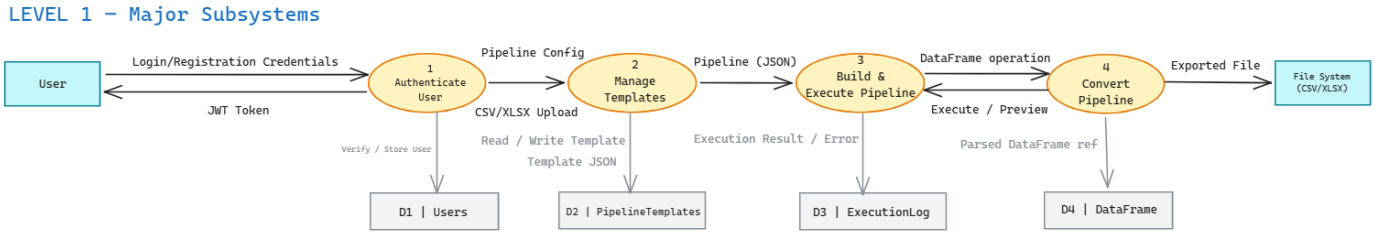


Figure 2: Level 1 – Major Subsystems

Processes & Data Hand-offs:

- **1. Authenticate User:** Takes the login or registration details a user enters, checks them against the saved account records in **D1 Users**, and sends back a login token confirming the user is who they say they are.
- **2. Manage Templates:** Once logged in, this process accepts the user's pipeline design or an uploaded data file. It reads and writes the pipeline's saved layout to and from **D2 PipelineTemplates**, then passes the pipeline along to be run.
- **3. Build & Execute Pipeline:** Receives the pipeline design and runs it step by step, producing either a live preview or an error. It records what happened (success or failure) in **D3 ExecutionLog**, and keeps track of the data currently being worked on in **D4 Working Data**.
- **4. Convert Pipeline:** Talks back and forth with the execution process to translate the visual pipeline into an Excel formula, and finally sends the completed, exported file out to the file system.

Level 2 – Process 3: Build & Execute Pipeline

This diagram zooms further into Process 3, showing exactly how a user's file or formula turns into a running pipeline, a live preview, and, if something goes wrong, an on-screen error.

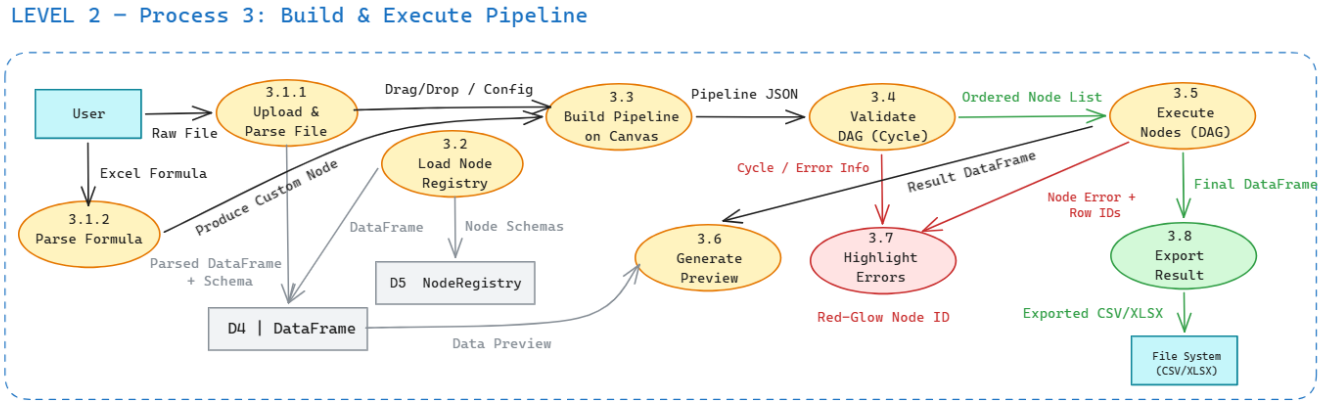


Figure 3: Level 2 – Process 3 Pipeline Execution

Execution Data Flow:

- **Bringing in data:** A user can either upload a raw data file, which is read and stored as a working table in **D4 Working Data**, or paste in an existing Excel formula, which is automatically turned into a ready-made building block on the canvas.
- **Building the pipeline:** The canvas itself is populated using the list of available building blocks kept in **D5 Node Registry**. The user drags, drops, and configures these blocks to build their pipeline.
- **Checking before running:** Before anything is executed, the finished pipeline is checked for problems such as circular connections. If it passes, an ordered list of steps is produced; if not, the specific problem is flagged.
- **Running the pipeline:** The steps are carried out in order, using the working data from **D4**, producing a result table.
- **Showing results and errors:** If everything succeeds, a preview of the result is shown to the user. If a step fails, the exact block and the exact rows responsible are identified so the problem block can be highlighted in red on the canvas.
- **Exporting:** Once a pipeline runs successfully all the way through, the final result can be exported and sent out as a downloadable data file.

Level 2 – Process 4: Convert Pipeline

This diagram shows how a visual pipeline is translated into an Excel formula that the user can read, copy, and reuse outside FLUX.

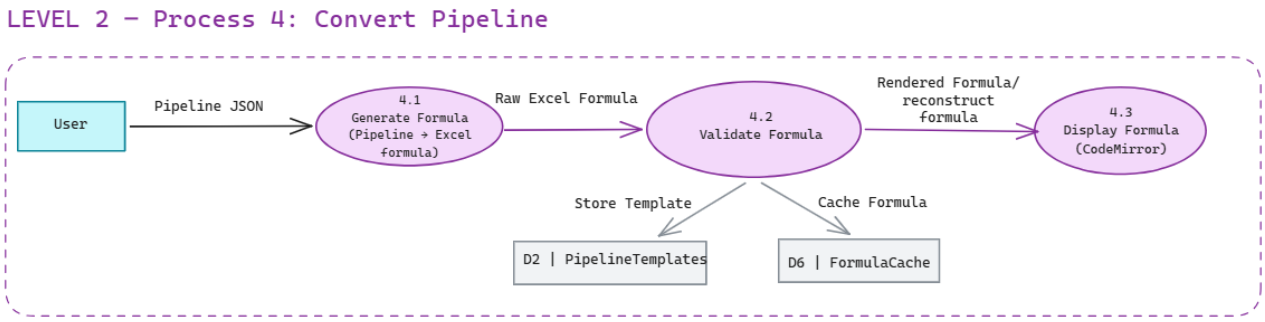


Figure 4: Level 2 – Process 4 Formula Conversion

Formula Conversion & Display Flow:

- **Generating the formula:** The user's pipeline design is sent in, and each block in it is translated, in order, into a single combined Excel formula.
- **Checking and saving the formula:** The generated formula is checked to make sure it is valid. Once confirmed, the pipeline itself can be saved as a reusable template in **D2 PipelineTemplates**, and the formula is also kept in **D6 Formula Cache** so it doesn't need to be rebuilt from scratch next time.
- **Showing the result:** The finished, readable formula is displayed back to the user, so they can copy it directly into a spreadsheet.

Complete Data Flow Diagram (All Levels)

The figure below brings together the Level 0 context diagram, the Level 1 subsystem breakdown, and both Level 2 process diagrams into a single, unified view of how data moves through FLUX end to end.

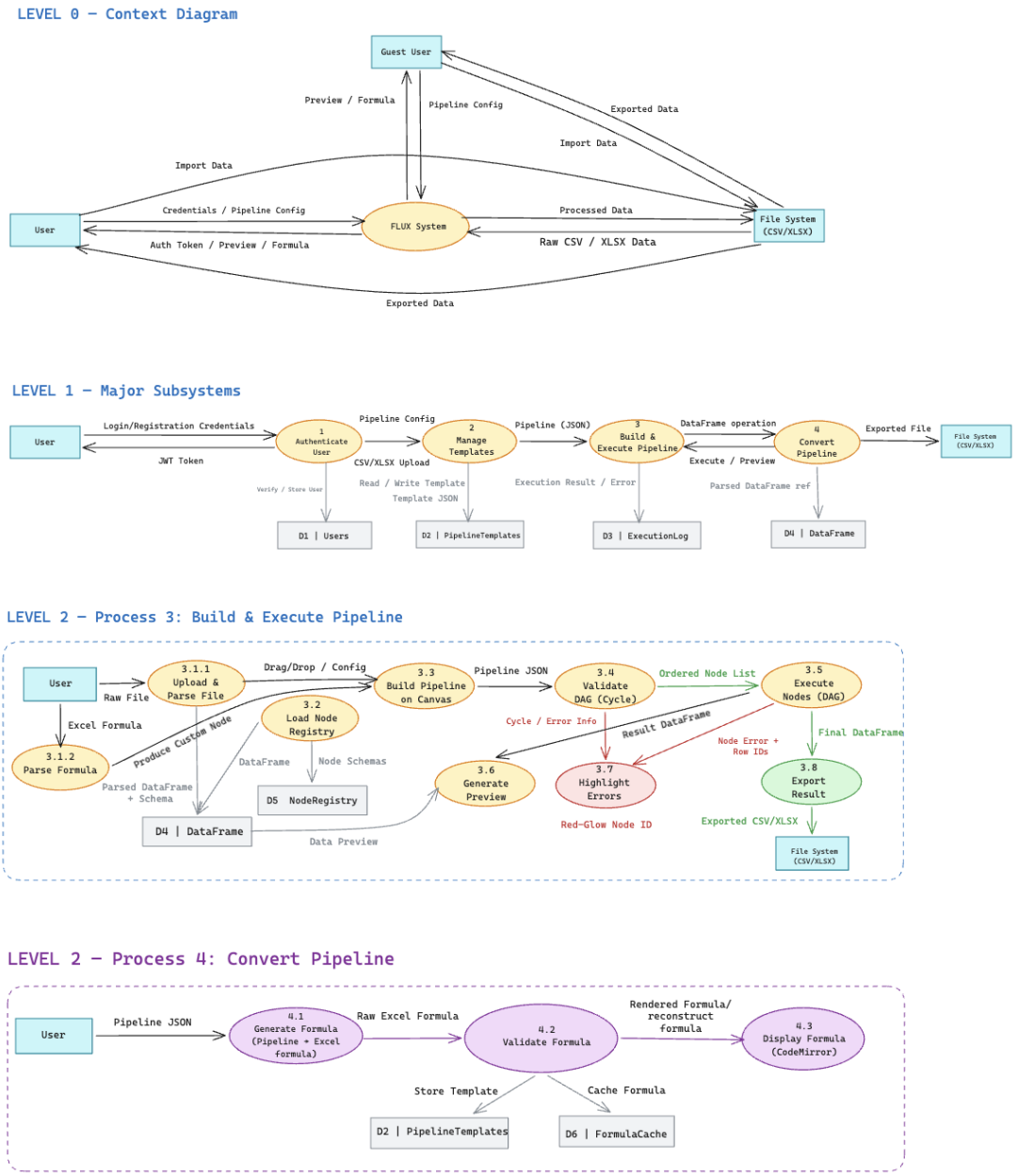


Figure 5: Complete Data Flow Diagram – Levels 0, 1, and 2

Three-Layered Architecture

FLUX is organized into three clear layers. Each layer only talks to the layer directly above or below it, which keeps the system easy to reason about: the user interface never touches the data directly, and the data never has to know how it will be displayed.

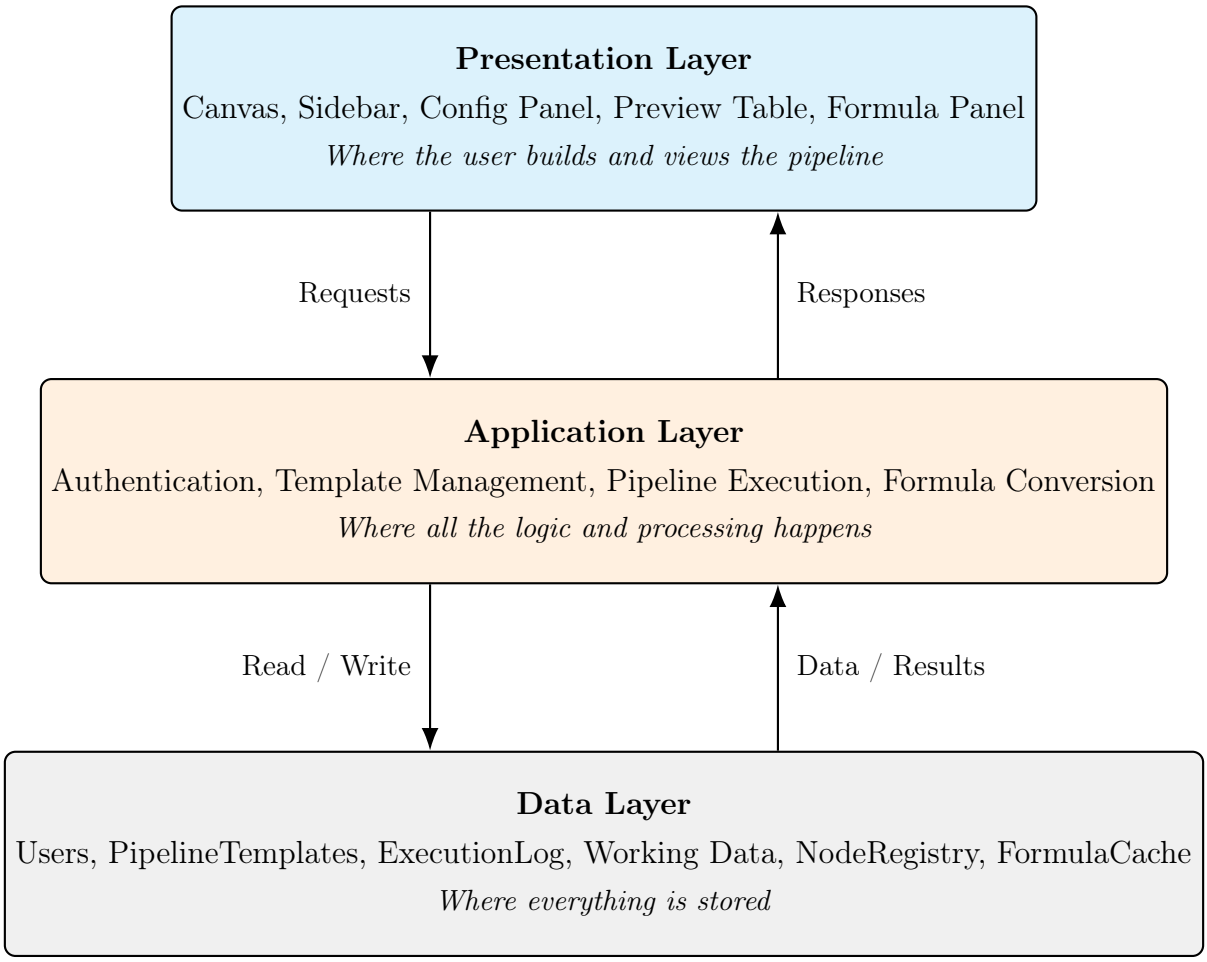


Figure 6: Three-Layered Architecture of FLUX

What each layer does:

- **Presentation Layer:** This is everything the user directly sees and interacts with – the drag-and-drop canvas, the sidebar of building blocks, the configuration panel, the live preview table, and the formula display. It sends the user's actions down as requests and shows whatever comes back.
- **Application Layer:** This is the "brain" of FLUX. It checks logins, manages saved pipelines, runs the pipeline step by step, and converts pipelines into Excel formulas. It never stores anything long-term itself – it always reads from or writes to the layer below.
- **Data Layer:** This is where everything is kept: user accounts, saved pipeline templates, run history, the data table currently being worked on, the catalog of available building blocks, and cached formulas. It simply stores and returns data whenever the Application Layer asks for it.

Appendix: Data Store Legend

ID	Store Role
D1 | Users	Keeps every user's account details and login information.
D2 | PipelineTemplates	Keeps saved pipeline designs (the blocks and connections a user built) so they can be reopened and reused later.
D3 | ExecutionLog	Keeps a record of past pipeline runs and whether each one succeeded or failed.
D4 | Working Data	Holds the data table currently being read, transformed, or previewed while a pipeline runs.
D5 | NodeRegistry	Holds the list of all available building blocks and their settings, used to populate the canvas.
D6 | FormulaCache	Stores already-generated Excel formulas so a repeated pipeline doesn't need to be re-translated.

References

- Software Requirements Specification (SRS) – FLUX: https://docs.google.com/spreadsheets/d/1-8UWCKcLwt3BBs4GePXok5uqgWQqSYcczDbF8JYtNFA/edit?gid=0#gid=0

Note: The Guest User role was introduced after feedback from the initial SRS presentation. A guest has access to every facility a registered user does, with the one exception that a guest cannot save their pipeline as a template.